HARI 3

# Messaging, Konsistensi Data, Security, dan Observability

Kafka, *transactional outbox*, saga dengan kompensasi di Lab 5, *idempotency*, OAuth2 dan JWT antar service, serta *observability* lintas service

DISIAPKAN UNTUK

**Peserta In-house Training**

**Endy Muhardin**

PT ArtiVisi Intermedia · 28 Oktober 2026

HARI 3

# Agenda hari ini

| Waktu | Topik | Format |
|---|---|---|
| 08:00 – 08:15 | Rekap Hari 2 dan penyamaan posisi dari *checkpoint* `lab-04` bagi peserta yang tertinggal | Diskusi |
| 08:15 – 09:15 | Peta *concern* Hari 3, komunikasi asinkron dengan Kafka, *at-least-once delivery*, *transactional outbox* | Presentasi + demo |
| 09:15 – 11:15 | **Lab 5**: transfer sebagai saga, event ke Kafka melalui outbox, notification sebagai *consumer* | *Hands-on* |
| 11:30 – 12:00 | *Idempotency*: *idempotent consumer* dan *Idempotency-Key* | Presentasi |
| 12:00 – 13:00 | Istirahat makan siang | — |
| 13:00 – 13:45 | **Lab 6**: *idempotent consumer* dan *Idempotency-Key* pada API transfer | *Hands-on* |
| 13:45 – 14:45 | OAuth2 *client credentials*, SSO dan *authorization server* (*embedded* atau *standalone*), validasi JWT, posisi API *gateway* | Presentasi + demo |
| 15:00 – 16:00 | **Lab 7**: *middleware* JWT pada REST dan *interceptor* JWT pada gRPC | *Hands-on* |
| 16:00 – 17:00 | *Observability* lintas service dan persiapan Lab 8. Ringkasan hari ini pada peta *concern* | Presentasi + demo |

PETA CONCERN MICROSERVICES

# *Concern* yang dibahas di Hari 3

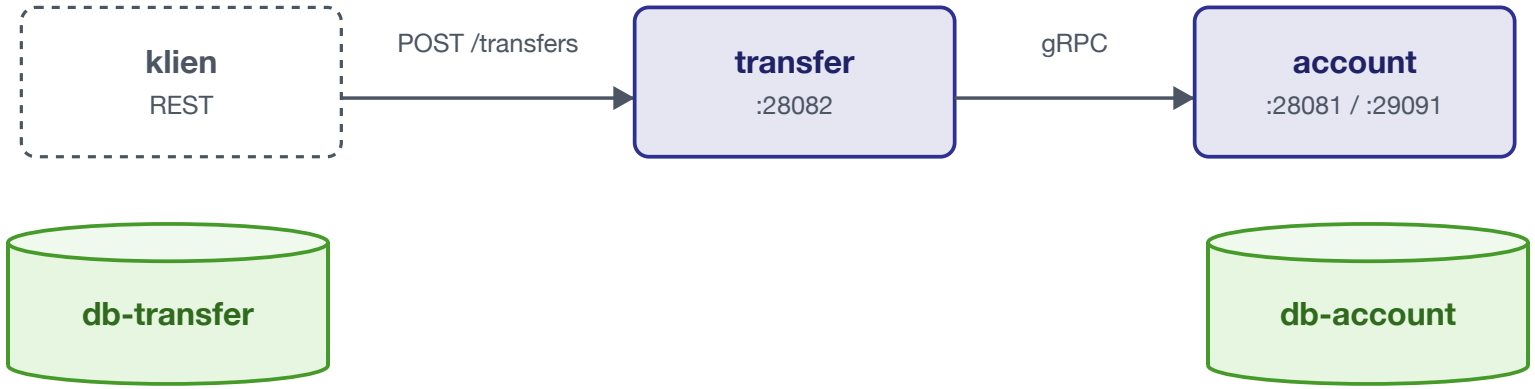
| # | *Concern* | Pertanyaan yang dijawab | Hari dan lab | Di kode (*library*) | Di platform atau infrastruktur |
|---|---|---|---|---|---|
| 1 | *Decomposition* dan kepemilikan data | *Service boundary* dan pemilik data | H1, Lab 1–2 | — | *Database per service* |
| **2** | **Komunikasi sinkron dan asinkron** | REST, gRPC, *messaging* | H1–H3, Lab 2, 4, 5 | `net/http`, grpc-go, kafka-go | Kafka |
| 3 | *Resiliency* | Tetap berfungsi saat dependensi lambat atau mati | H2, Lab 3–4 | failsafe-go | *Probe*, *restart*, *rolling update* (H4) |
| **4** | **Konsistensi data** | Perubahan data lintas database | H2–H3, Lab 5–6 | Saga, *outbox*, *idempotency* | — |
| **5** | ***Security*** | Identitas pengguna (SSO) dan service, serta hak akses | H3, Lab 7 | JWT *middleware* dan *interceptor* | *Authorization server* atau IdP (Hydra di lab, Keycloak), API *gateway* |
| **6** | ***Observability*** | Kondisi tiap service dan integrasi antarservice | H3–H4, Lab 8 | `slog`, Prometheus *client*, OTel SDK | Grafana, Loki, Tempo, Prometheus |
| 7 | *Configuration* (termasuk *centralized config* dan *secret*) | Nilai yang berbeda per *environment* | H1, Lab 1 (*env var*); H4, Lab 9 (ConfigMap, Secret) | `LoadConfig` | ConfigMap, Secret, *secret manager* |
| 8 | *Service discovery* dan *load balancing* | Alamat service tujuan | H4, Lab 9 | — (alamat dari konfigurasi) | Kubernetes Service (DNS), Ingress |
| 9 | *Packaging* dan *deployment* | Cara service dijalankan dan diperbarui | H4, Lab 9 | Dockerfile | Kubernetes Deployment, *probe*, *rolling update* |
| 10 | *Automation* (CI/CD) | Dari *commit* ke *production* | H4, Lab 10 | Target `Makefile` | *Pipeline* (git server dan runner CI, Jenkins) |

**CATATAN**

Baris berwarna dibahas hari ini. Sebagian besar jawabannya ada di kode setiap service: saga, outbox, *idempotency*, dan validasi JWT. Platform menyediakan Kafka, Hydra, dan *backend observability*. Untuk *observability*, hari ini membahas konsep dan kode instrumentasi yang dipasang di Lab 8. Lab 8 dikerjakan di awal Hari 4.

POSISI STUDI KASUS

# Kondisi sistem di akhir Hari 2




## Sudah ada

- Panggilan transfer ke account melalui gRPC
- *Timeout*, *retry*, dan *circuit breaker* di *interceptor* klien

## Celah yang ditutup hari ini

- Bila debit berhasil tetapi credit gagal, saldo rekening sumber berkurang tanpa dikembalikan
- Debit belum *idempotent*, jadi *timeout* belum boleh di-*retry*
- Siapa pun di jaringan internal dapat memanggil Debit

**CATATAN**

Desain saga, kompensasi, dan mutasi *idempotent* sudah dibahas di akhir Hari 2. Lab 5 hari ini mengimplementasikannya bersama outbox dan Kafka.

# 01

# Komunikasi asinkron dengan Kafka

*Topic*, *partition*, *consumer group*, dan jaminan pengiriman

KONSEP

# Panggilan sinkron dan asinkron

## Sinkron (REST, gRPC)

- Pemanggil menunggu jawaban
- Penerima harus aktif saat dipanggil
- Cocok untuk langkah yang hasilnya langsung dibutuhkan, misalnya debit
- Kegagalan langsung terlihat oleh pemanggil

## Asinkron (event melalui *broker*)

- Pengirim menulis event lalu melanjutkan proses
- Penerima boleh mati sementara, event tetap tersimpan di *broker*
- Cocok untuk reaksi atas fakta yang sudah terjadi, misalnya notifikasi
- Event bisa terkirim lebih dari sekali

**CATATAN**

Debit dan credit tetap sinkron karena hasilnya menentukan status transfer. Alasan notifikasi dikirim secara asinkron ada di *slide* berikutnya.

KEPUTUSAN DESAIN

# *Messaging* untuk notifikasi transfer

## Kebutuhan

- Nasabah menerima notifikasi untuk setiap transfer yang selesai atau gagal
- Gangguan di notification tidak boleh menggagalkan transfer
- Service lain, misalnya pencatatan audit, dapat ikut bereaksi atas transfer tanpa mengubah transfer

## Opsi yang dipertimbangkan

- Transfer memanggil notification secara sinkron (REST atau gRPC) setelah saga selesai
- Transfer menulis event ke *message broker*, lalu notification membacanya sebagai *consumer*

## Pilihan di studi kasus

Event ke *topic* `transfer-events`, dibaca oleh *consumer group* `notification`:

- *Temporal decoupling*: notification boleh mati, event menunggu di *broker*
- *Load leveling*: lonjakan transfer ditampung *broker*, notification memproses sesuai kapasitasnya
- *Fan-out*: *consumer* baru cukup membuat *consumer group* sendiri

## Konsekuensi dan *trade-off*

- *Eventual consistency*: notifikasi tiba sesaat setelah status transfer berubah, bukan bersamaan
- Duplikat dan urutan event harus ditangani penerima (*key* `transfer_id`, Lab 6)
- *Broker* menjadi infrastruktur baru yang harus dioperasikan dan dipantau
- Alur lintas *broker* hanya terlihat utuh melalui *tracing* (Lab 8)

KEPUTUSAN DESAIN

# Kafka dibandingkan RabbitMQ

## Kebutuhan

- Event transfer dapat dibaca ulang (*replay*) setelah *bug* di *consumer* diperbaiki
- Urutan event milik satu transfer terjaga
- Beberapa *replica* notification berbagi beban

## Opsi yang dipertimbangkan

| Aspek | Kafka | RabbitMQ (*classic* dan *quorum queue*) |
|---|---|---|
| Penyimpanan | *Log* yang hanya ditambah, dibaca berdasarkan *offset* | *Message* dihapus dari *queue* setelah *consumer* mengirim *ack* |
| *Replay* | *Offset* dikembalikan, selama masih dalam masa retensi | *Producer* harus mengirim ulang |
| Urutan | Per *partition* | Per *queue* (FIFO), dapat berubah bila terjadi *redelivery* dan ada beberapa *consumer* |
| *Routing* | Berdasarkan *topic* dan *key* | *Exchange* direct, topic, fanout, headers |

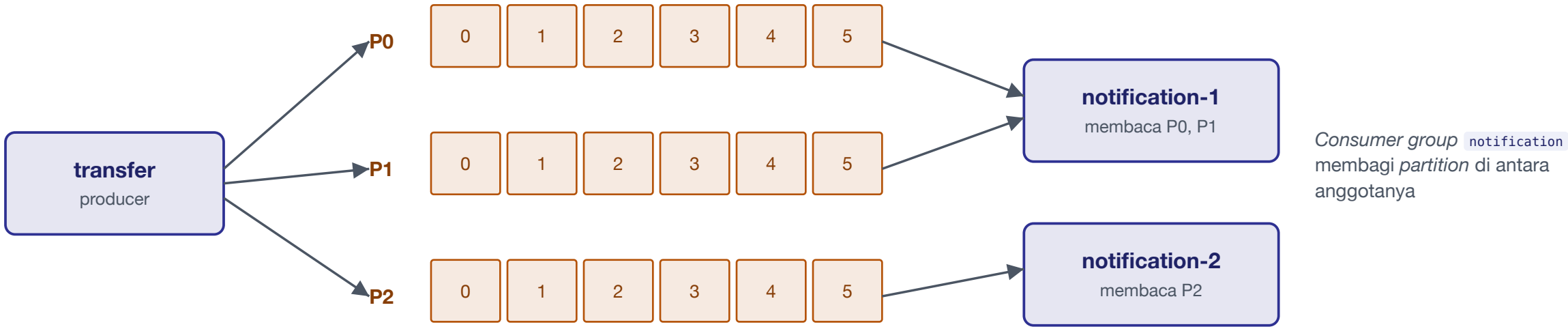
## Pilihan di studi kasus

Kafka (*mode* KRaft, tanpa ZooKeeper) dengan *client* kafka-go. Retensi dan *replay* digunakan langsung di Lab 5 dan Lab 6 (`make replay`), urutan dijaga dengan *key* `transfer_id`

## Konsekuensi dan *trade-off*

- Jumlah *consumer* aktif per *group* dibatasi jumlah *partition* (3 di studi kasus)
- Pilihan *routing* lebih sedikit dibanding *exchange* RabbitMQ, sehingga penyaringan event dilakukan di *consumer*
- Lab menggunakan satu *broker* tanpa replikasi, sedangkan *production* butuh beberapa *broker* dan kebijakan retensi
- RabbitMQ Streams juga menyediakan *log* dengan *replay*. Tabel ini membandingkan *classic* dan *quorum queue*

# *Topic*, *partition*, dan *offset*




*Consumer group* `notification` membagi *partition* di antara anggotanya

- *Topic* `transfer-events` terdiri atas 3 *partition*, dan setiap *partition* berupa *log* yang hanya ditambah di ujungnya
- *Offset* menandai posisi baca *consumer group* di setiap *partition*, dan *broker* menyimpannya per *group*
- Satu *partition* hanya dibaca oleh satu anggota *group*, sehingga jumlah anggota yang aktif maksimal sama dengan jumlah *partition*

KAFKA

# *Key* menentukan *partition* dan urutan

- *Producer* mengirim *message* dengan *key* `transfer_id`
- Semua *message* dengan *key* yang sama masuk ke *partition* yang sama, asalkan `kafka.Writer` diberi `Balancer: &kafka.Hash{}`. Tanpa pengaturan ini, kafka-go menggunakan *balancer default round-robin* yang mengabaikan *key*
- Urutan hanya dijamin di dalam satu *partition*, tidak di seluruh *topic*
- Akibatnya, event milik transfer yang sama selalu dibaca berurutan

### Isi satu *message*

| Bagian | Isi |
|---|---|
| *Key* | `transfer_id` |
| *Header* | `event_type`, `event_id` |
| *Value* | *Payload* JSON event |

**PERHATIAN**

Tanpa *key* dan tanpa `kafka.Hash`, event untuk satu `transfer_id` tersebar ke beberapa *partition*. Saat ini setiap transfer menghasilkan satu event akhir, tetapi urutan per transfer menjadi penting begitu satu transfer menghasilkan lebih dari satu event, misalnya `TransferDebited` lalu `TransferCompleted`.

KAFKA

# Tiga tingkat jaminan pengiriman

| Jaminan | Cara kerja di sisi *consumer* | Akibat saat *consumer* mati di tengah proses |
|---|---|---|
| *At-most-once* | *Commit offset* terlebih dahulu, lalu proses | Event hilang dan tidak pernah diproses |
| *At-least-once* | Proses terlebih dahulu, lalu *commit offset* | Event diproses ulang, sehingga muncul duplikat |
| *Exactly-once* | Transaksi Kafka, hanya berlaku dari Kafka ke Kafka | Tidak mencakup penulisan ke PostgreSQL milik service |

**CATATAN**

Studi kasus memilih *at-least-once*. Kehilangan notifikasi transfer tidak dapat diterima, sedangkan duplikat dapat dicegah di sisi penerima dengan *idempotent consumer* (Lab 6).

# *Consumer* dengan kafka-go

```go
r := kafka.NewReader(kafka.ReaderConfig{
    Brokers: cfg.KafkaBrokers,
    Topic:   cfg.KafkaTopic,
    GroupID: "notification",
})

for {
    m, err := r.FetchMessage(ctx)
    if err != nil {
        return fmt.Errorf("fetch: %w", err)
    }
    if err := h.Handle(ctx, m); err != nil {
        return fmt.Errorf("handle %s: %w", m.Key, err)
    }
    if err := r.CommitMessages(ctx, m); err != nil {
        return fmt.Errorf("commit: %w", err)
    }
}
```

Sumber: `notification/internal/notification/consumer.go` · *tag* `lab-05`

- `FetchMessage` tidak melakukan *commit* otomatis
- *Offset* di-*commit* setelah baris notifikasi tersimpan di database
- Bila *consumer* mati sebelum *commit*, event yang sama dibaca lagi setelah *restart*
- Di Lab 5 *handler* ini sengaja belum *idempotent*

# 02

# *Transactional outbox*

Status di database dan event di Kafka harus selalu sejalan

MASALAH

# *Dual write* ke database dan Kafka

### Tulis DB dulu, lalu *publish*

```
tx.Commit(ctx)          // status COMPLETED
w.WriteMessages(ctx, m) // proses mati di sini
```

Transfer selesai, tetapi event tidak pernah terkirim. Nasabah tidak menerima notifikasi.

### *Publish* dulu, lalu tulis DB

```
w.WriteMessages(ctx, m) // event terkirim
tx.Commit(ctx)          // gagal
```

Notifikasi "transfer berhasil" terkirim, padahal status transfer di database tidak berubah.

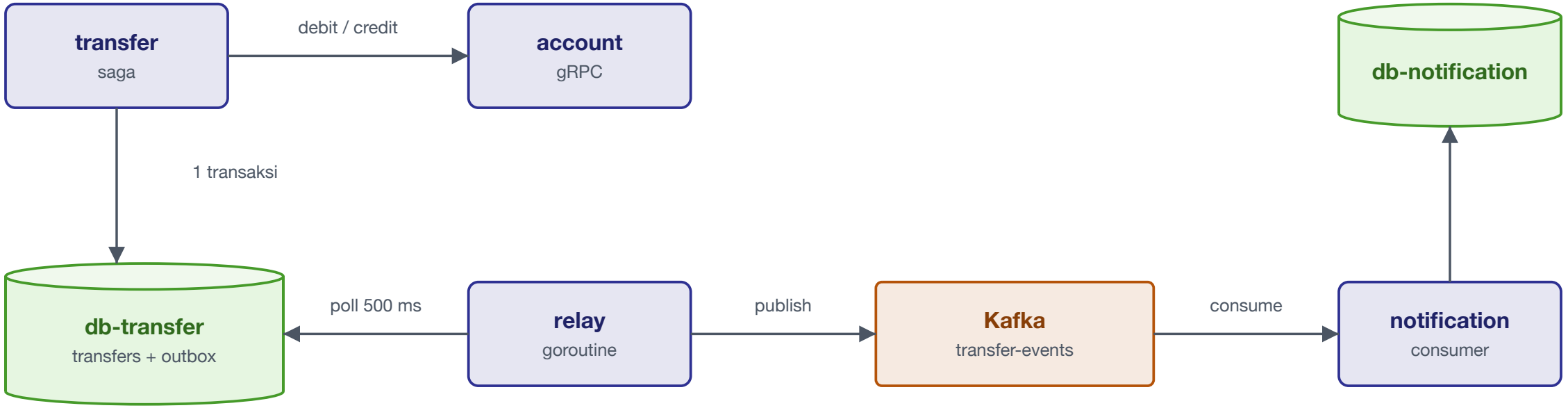
Ilustrasi, tidak diambil dari repositori studi kasus.

**CATATAN**

Database dan Kafka tidak bisa digabung dalam satu transaksi. Urutan mana pun tetap menyisakan celah ketika proses mati di antara dua langkah tersebut.

POLA

# Alur *transactional outbox*




- Perubahan status transfer dan baris outbox ditulis dalam **satu transaksi database**, sehingga keduanya tersimpan bersama atau batal bersama
- *Relay* membaca outbox secara terpisah lalu mengirim ke Kafka

# Status dan event dalam satu transaksi

```go
func (s *PostgresStore) Transition(ctx context.Context,
    id string, from, to Status, reason string,
    event EventType) (Transfer, error) {
    tx, err := s.pool.Begin(ctx)
    // ...
    defer tx.Rollback(ctx) // no-op after Commit

    t, err := one(ctx, tx, `
        update transfer set status = @to, ...
        where id = @id and status = @from
        returning `+columns, pgx.NamedArgs{...})
    // ... tidak ada baris: ErrStaleState
    if event != "" {
        // e := Event{EventID: uuid.NewString(), ...}
        payload, err := json.Marshal(e)
        // ...
        err = outbox.Insert(ctx, tx, outbox.Message{
            ID: e.EventID, AggregateID: t.ID,
            EventType: string(event), Payload: payload})
        // ...
    }
    return t, tx.Commit(ctx)
}
```

Sumber: `transfer/internal/transfer/store.go` · *tag* `lab-05`

- `tx.Rollback` setelah `Commit` tidak berpengaruh, jadi aman di-`defer`
- `and status = @from` menjaga urutan status, sehingga transfer yang statusnya sudah diubah proses lain tidak ditimpa (`ErrStaleState`)
- Tabel `outbox`: `id`, `aggregate_id`, `event_type`, `payload` (jsonb), `created_at`, `published_at`
- Event hanya untuk status akhir: `TransferCompleted`, `TransferFailed`, `TransferCompensated`

# *Relay* dari outbox ke Kafka

```
rows, err := tx.Query(ctx, `
    select id::text, aggregate_id::text, event_type, payload
    from outbox
    where published_at is null
    order by created_at
    limit @limit
    for update skip locked`,
    pgx.NamedArgs{"limit": BatchSize}) // 100
// ... baca baris menjadi msgs ([]kafka.Message)

if err := r.pub.WriteMessages(pubCtx, msgs...); err != nil {
    return 0, fmt.Errorf("publish %d events: %w",
        len(msgs), err)
}
// jendela duplikat: event sudah di Kafka,
// published_at belum tersimpan
_, err = tx.Exec(ctx, `update outbox
    set published_at = now()
    where id = any(@ids::uuid[])`,
    pgx.NamedArgs{"ids": ids})
```

Sumber: `transfer/internal/outbox/outbox.go` · *tag* `lab-05`

- Dengan `for update skip locked`, beberapa *replica* transfer dapat menjalankan *relay* tanpa mengambil baris yang sama
- *Key* `transfer_id` dengan `Balancer: &kafka.Hash{}` di `kafka.Writer`, *header* `event_type` dan `event_id`
- Bila proses mati setelah *publish* dan sebelum `published_at` diisi, event dikirim ulang
- Karena itu, outbox hanya menjamin *at-least-once delivery*

KONTRAK EVENT

# *Payload* event transfer

```
{
  "event_id":     "8f0c...e1",
  "transfer_id":  "3b2a...9d",
  "from_account": "1001",
  "to_account":   "1002",
  "amount":       "250000.00",
  "status":       "COMPLETED",
  "reason":       "",
  "occurred_at":  "2026-10-28T10:15:00Z"
}
```

- `event_id` unik per event, menjadi dasar deteksi duplikat di Lab 6
- `amount` berupa *string* desimal, konsisten dengan kontrak protobuf Hari 2
- *Payload* memuat semua data yang dibutuhkan notification, sehingga notification tidak perlu memanggil transfer
- Event menyatakan fakta yang sudah terjadi, karena itu nama event menggunakan bentuk *past tense* bahasa Inggris, misalnya `TransferCompleted`

CONSUMER

# *Event* yang tidak mungkin diproses (*poison message*)

### Dicatat `ERROR` lalu dilewati

- JSON yang tidak dapat dibaca
- `event_id` atau `transfer_id` yang bukan UUID 36 karakter
- Ditolak PostgreSQL dengan SQLSTATE kelas 22 (data tidak valid, misalnya `22P02`) atau 23 (*constraint*, misalnya `23502`)
- *Offset* tetap di-*commit*, sehingga event berikutnya di *partition* yang sama tidak tertahan

### Diulang sampai berhasil

- Database atau Kafka mati, koneksi putus, *timeout*
- *Error* skema (kelas 42, misalnya tabel tidak ada) membuat semua event gagal dengan cara yang sama. Melewatinya berarti membuang semua notifikasi, sedangkan mengulangnya hanya menunda pemrosesan event hingga perbaikan di-*deploy*

**CATATAN**

Event yang terus gagal tanpa batas memblokir seluruh *partition*. Di *production*, event seperti ini dikirim ke *dead-letter topic* untuk diperiksa petugas.

LAB 5

# Transfer sebagai saga dengan outbox dan Kafka

## Tujuan

Transfer yang gagal di tengah jalan selalu berakhir di status yang jelas, dan setiap perubahan status menghasilkan event yang sampai ke notification.

## Yang dikerjakan

- Kafka dan db-notification di `compose.yaml`
- Mutasi account *idempotent* berdasarkan *reference*
- Saga di transfer, termasuk *reversal*
- Outbox dan *relay* ke *topic* `transfer-events`
- Service notification sebagai *consumer*

- Langkah pertama: `make infra-reset` lalu `make infra-up`. Kafka baru muncul di lab ini dan *unique index* mutasi membutuhkan database kosong
- Uji kompensasi: transfer ke `to_account` yang tidak terdaftar menghasilkan `COMPENSATED` dan dua mutasi di db-account. Dengan `make fault-error RATE=0.6` dan ±20 transfer bersamaan, sebagian transfer menampilkan log `retrying reversal` lalu berakhir `COMPENSATED` dengan HTTP `503`
- Uji duplikat: hentikan notification, jalankan `make replay`, lalu lihat notifikasi ganda

**TIPS**

Langkah lengkap ada di Panduan Lab, bab Lab 5. Lab ini dimulai dari tag `lab-04` dan berakhir di tag `lab-05`.

# 03

## *Idempotency*

Pemrosesan ulang tidak boleh menggandakan mutasi

MASALAH

# Sumber duplikat dalam studi kasus

| Sumber | Kejadian |
|---|---|
| *Relay* outbox | Proses mati setelah *publish*, sebelum `published_at` tersimpan |
| *Consumer* Kafka | Proses mati setelah menulis ke database, sebelum *commit offset* |
| *Rebalance consumer group* | *Partition* pindah ke anggota lain sebelum *offset* terakhir di-*commit* |
| *Replay* | *Offset* sengaja dikembalikan ke awal untuk memproses ulang |
| Klien API | Aplikasi kanal mengirim ulang `POST /transfers` karena *timeout* |

**CATATAN**

Duplikat tidak bisa dicegah di sisi pengirim. Karena itu, sisi penerima harus mengenali permintaan yang sudah pernah diproses.

LAB 6

# *Idempotent consumer* di notification

```go
func (s *PostgresStore) Insert(ctx context.Context,
    n Notification) (Notification, bool, error) {
    tx, err := s.pool.Begin(ctx)
    if err != nil {
        return Notification{}, false, err
    }
    defer tx.Rollback(ctx) // no-op after Commit

    tag, err := tx.Exec(ctx, `insert into processed_event
        (event_id) values (@event_id)
        on conflict (event_id) do nothing`,
        pgx.NamedArgs{"event_id": n.EventID})
    if err != nil {
        return Notification{}, false, err
    }
    if tag.RowsAffected() == 0 {
        return Notification{}, false, nil // duplikat
    }
    rows, err := tx.Query(ctx, `insert into notification
        (event_id, transfer_id, event_type, message)
        values (...) returning `+columns, ...)
    // ... stored dari rows
    return stored, true, tx.Commit(ctx)
}
```

Sumber: `notification/internal/notification/store.go` · *tag* `lab-06`

- Tabel `processed_event` dengan `event_id` sebagai *primary key*
- Tanda "sudah diproses" dan notifikasi ditulis dalam satu transaksi
- Untuk duplikat, *consumer* menerima `inserted` = `false`, mencatat log `duplicate event skipped`, lalu melewati event tersebut, tetapi *offset* tetap di-*commit*
- Setelah ini, `make replay` tidak lagi menghasilkan notifikasi ganda, dan `make duplicates` menampilkan daftar kosong

LAB 6

# *Idempotency-Key* pada `POST /transfers`

```
POST /transfers
Idempotency-Key: 7d1e4c2a-...
Content-Type: application/json

{"from_account":"1001",
 "to_account":"1002",
 "amount":"250000.00"}
```

| Permintaan | Hasil |
|---|---|
| *Key* baru | Transfer baru, saga dijalankan |
| *Key* sama, *body* sama | Transfer asli, kode HTTP sama dengan respons pertama, *header* `Idempotent-Replayed: true` |
| *Key* sama, transfer pertama masih berjalan | `202`, periksa `GET /transfers/{id}` |
| *Key* sama, *body* berbeda | `422` |
| Format *key* salah | `400` |

- *Key* dibuat oleh aplikasi kanal dan disimpan dengan *unique index* di db-transfer. *Replay* transfer yang selesai menjawab `201`, bukan `200`
- Karena kode HTTP-nya sama dengan respons pertama, klien yang kehilangan respons pertama tetap mengambil keputusan yang sama. Penyebab `FAILED` dan `COMPENSATED` dibaca ulang dari teks `failure_reason`, sedangkan desain *production* menyimpannya di kolom kode tersendiri
- Baris `PENDING` atau `DEBITED` yang tertinggal karena *crash* tidak dilanjutkan, sehingga *replay*-nya terus menjawab `202`

RINGKASAN

# Tiga titik *idempotency* dalam studi kasus

| Titik | Penanda keunikan | Penyimpanan | Hasil saat duplikat |
|---|---|---|---|
| Mutasi account (Lab 5) | `account_number`, `type`, `reference` | *Unique constraint* di tabel mutasi | Saldo saat ini, tanpa mutasi baru |
| *Consumer* notification (Lab 6) | `event_id` | Tabel `processed_event` | Dilewati dan dicatat di log |
| API transfer (Lab 6) | *Header* `Idempotency-Key` | Kolom unik di tabel transfer | Transfer asli dengan kode HTTP respons pertama, atau `422` bila *body* berbeda |

TIPS

Penanda keunikan dan perubahan data selalu ditulis dalam satu transaksi database. Bila ditulis terpisah, celah duplikatnya hanya berpindah tempat.

LAB 6

# *Replay* tanpa notifikasi ganda dan tanpa mutasi saldo ganda

### Tujuan

Event dan perintah mutasi yang sama boleh diterima berkali-kali tanpa menghasilkan notifikasi ganda atau mutasi saldo ganda, dan permintaan transfer yang dikirim ulang tidak membuat transfer kedua.

### Yang dikerjakan

- Tabel `processed_event` di db-notification
- *Handler* notification yang *idempotent*
- *Header* `Idempotency-Key` di `POST /transfers`
- *Test* untuk duplikat dan *body* berbeda

1. Hentikan notification, tunggu 10–30 detik hingga sesi *consumer group* berakhir, jalankan `make replay`, lalu jalankan notification kembali. `make replay` yang terlalu cepat gagal dengan pesan `group ... is Stable`
2. `make duplicates` menampilkan daftar kosong, artinya tidak ada event dengan lebih dari satu notifikasi
3. Kirim `POST /transfers` dua kali dengan *key* yang sama. Jawaban kedua berupa `201` dengan `Idempotent-Replayed: true`, dan saldo kedua rekening hanya berubah satu kali. Bila *body* berbeda, jawabannya `422`

**TIPS**

Langkah lengkap ada di Panduan Lab, bab Lab 6. Lab ini dimulai dari tag `lab-05` dan berakhir di tag `lab-06`.

# 04

# Security antar service

OAuth2 *client credentials*, JWT, *middleware* REST, dan *interceptor* gRPC

MASALAH

# Jaringan internal tidak otomatis aman

- Saat ini setiap proses yang bisa menjangkau port 29091 dapat memanggil Debit
- Satu service yang disusupi membuka akses ke semua service lain
- Account perlu tahu **siapa** pemanggilnya dan **apa** yang boleh dilakukan pemanggil tersebut

### Yang dibutuhkan

- Identitas untuk setiap service (*client*)
- Hak akses terbatas per operasi (*scope*)
- Bukti identitas yang dapat diperiksa tanpa memanggil server pusat di setiap *request*

**CATATAN**

Prinsip ini disebut *zero trust*. Setiap panggilan wajib diperiksa, termasuk panggilan dari dalam jaringan sendiri.

KEPUTUSAN DESAIN

# OAuth2 *client credentials* dengan Hydra

## Kebutuhan

- Identitas per service dan hak akses per operasi (*scope*)
- Token diperiksa secara lokal di setiap service
- Mengikuti standar terbuka, sehingga *authorization server* dapat diganti dengan milik organisasi

## Opsi yang dipertimbangkan

- *API key* atau *shared secret* per pasangan service: tanpa *scope* standar, rotasi manual
- mTLS: identitas di tingkat koneksi, butuh PKI, *scope* tetap diatur sendiri
- OAuth2 *client credentials* + JWT dari Hydra, Keycloak, atau IdP lain

## Pilihan di studi kasus

OAuth2 *client credentials* dengan Ory Hydra, dijalankan dari konfigurasi `compose.yaml` yang sudah disediakan. Hydra hanya berperan sebagai *authorization server* OAuth2/OIDC, tanpa UI *login* dan tanpa manajemen *user*. Untuk alur dengan *user*, *login* dan *consent* diserahkan ke aplikasi terpisah. *Client credentials* tidak melibatkan *user*, sehingga cukup satu *container*

## Konsekuensi dan *trade-off*

- Pengambilan token bergantung pada Hydra. Bila Hydra mati saat token harus diperbarui, transfer berakhir `FAILED` (saat debit) atau `COMPENSATION_FAILED` (saat *reversal*)
- Mengganti dengan Keycloak atau IdP lain cukup dengan mengubah `OAUTH_ISSUER`, `OAUTH_JWKS_URL`, dan `OAUTH_TOKEN_URL`, misalnya *issuer* `https://sso.example/realms/<realm>` dan JWKS `.../protocol/openid-connect/certs`
- Nama *claim scope* berbeda: Hydra `scp` (*array*), Keycloak `scope` (*string* dipisah spasi). *Struct* `Claims` di paket `auth` perlu disesuaikan

# *Authorization server*: *embedded* atau *standalone*

| Produk | Jenis | Bahasa | Lisensi | Situs |
|---|---|---|---|---|
| Spring Security (*Authorization Server*) | *Embedded* | Java | Apache-2.0 | `spring.io/projects/spring-security` |
| zitadel/oidc (*package* `op`) | *Embedded* | Go | Apache-2.0 | `github.com/zitadel/oidc` |
| node-oidc-provider | *Embedded* | Node.js | MIT | `github.com/panva/node-oidc-provider` |
| Laravel Passport | *Embedded* | PHP | MIT | `laravel.com/docs/passport` |
| Keycloak | *Standalone* | Semua | Apache-2.0 | `keycloak.org` |
| Ory Hydra (digunakan di lab) | *Standalone* | Semua | Apache-2.0 | `ory.sh/hydra` |
| ZITADEL | *Standalone* | Semua | AGPL-3.0 | `zitadel.com` |

### *Embedded*: *library* di dalam aplikasi

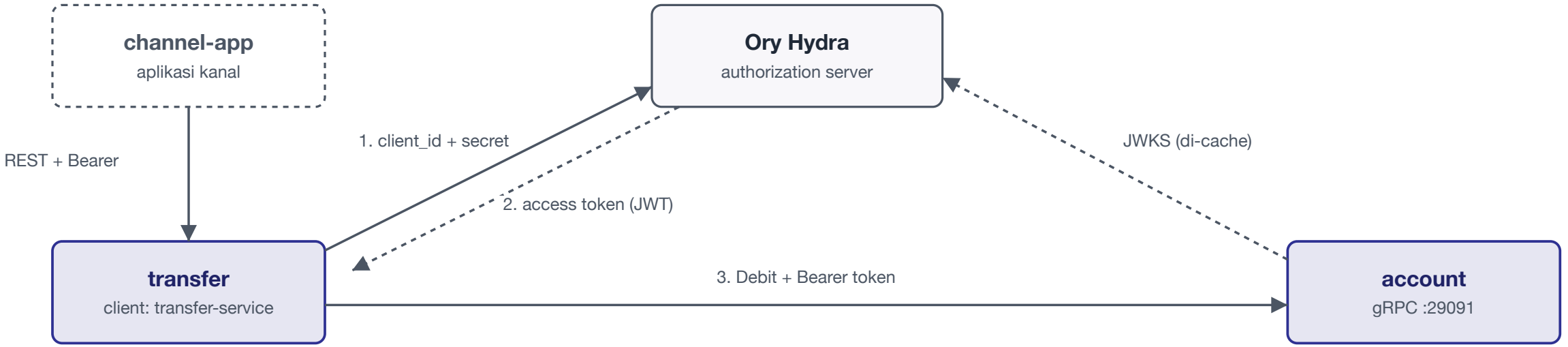
Satu artefak untuk di-*deploy*, tetapi terikat pada satu *tech stack*, dan tim memelihara kode *security* tersebut.

### *Standalone*: produk terpisah

Dapat digunakan service dalam bahasa apa pun dan siap dijalankan, tetapi menambah satu server yang harus dioperasikan.

SSO untuk pengguna (login aplikasi channel atau karyawan) dan *client credentials* untuk service menggunakan *authorization server* yang sama. Studi kasus hanya menggunakan *client credentials* dengan Hydra, karena tidak ada login pengguna. Hydra tidak menyediakan halaman login dan manajemen pengguna, sehingga SSO dengan Hydra membutuhkan aplikasi login terpisah.

# Alur *client credentials*




- Alur ini tidak melibatkan *login* pengguna. Service mengautentikasi dirinya sendiri dengan `client_id` dan `client_secret`
- Account memeriksa token secara lokal dengan *public key* dari JWKS, tanpa memanggil Hydra di setiap *request*

# Isi *access token* dari Hydra

```
// header
{ "alg": "RS256", "kid": "a1b2...", "typ": "JWT" }

// payload (claims)
{
  "iss": "http://localhost:14444/",
  "sub": "transfer-service",
  "client_id": "transfer-service",
  "scp": ["account:write"],
  "iat": 1793182500,
  "exp": 1793186100
}
```

| Bagian Token | Diperiksa untuk |
|---|---|
| `kid` (*header*) | Memilih *public key* dari JWKS |
| `iss` | Token berasal dari *issuer* yang dipercaya |
| `exp` | Token belum kedaluwarsa |
| `scp` | Pemanggil memiliki *scope* untuk operasi ini |

**PERHATIAN**

JWT hanya di-*sign*, tidak dienkripsi. Siapa pun yang memegang token dapat membaca isinya, jadi data rahasia tidak boleh dimasukkan ke *claim*.

# Validasi token dengan keyfunc dan golang-jwt

```go
keys, err := keyfunc.NewDefaultOverrideCtx(ctx,
    []string{jwksURL}, keyfunc.Override{
        RefreshUnknownKID: rate.NewLimiter(
            rate.Every(5*time.Second), 1),
        // ...
    })

parser := jwt.NewParser(
    jwt.WithValidMethods([]string{"RS256"}),
    jwt.WithIssuer(issuer),
    jwt.WithExpirationRequired(),
    jwt.WithLeeway(30*time.Second),
)

func (v *Verifier) Verify(token string) (Claims, error) {
    var c Claims
    if _, err := v.parser.ParseWithClaims(token, &c,
        v.keys.Keyfunc); err != nil {
        return Claims{}, fmt.Errorf("%w: %w",
            ErrInvalidToken, err)
    }
    return c, nil
}
```

Sumber: `transfer/internal/auth/verifier.go` · *tag* `lab-07`

- keyfunc mengambil JWKS saat *startup* dan setiap jam. Token dengan `kid` yang belum dikenal memicu pengambilan JWKS baru, paling sering sekali per 5 detik
- *Signature*, `iss`, dan `exp` diperiksa oleh *parser* `jwt`
- `WithValidMethods` menolak token dengan `alg` lain, termasuk `none`
- *Scope* diperiksa oleh pemanggil `Verify`, karena setiap operasi membutuhkan *scope* berbeda

LAB 7

# *Middleware* JWT di REST transfer

```go
func (m *Middleware) Require(scope string,
    next http.Handler) http.Handler {
    return http.HandlerFunc(func(
        w http.ResponseWriter, r *http.Request) {
        token, ok := bearer(r.Header.Get("Authorization"))
        if !ok {
            w.Header().Set("WWW-Authenticate", `Bearer`)
            writeError(w, http.StatusUnauthorized,
                ErrNoToken.Error())
            return
        }
        claims, err := m.v.Verify(token)
        if err != nil { // detail hanya di log
            writeError(w, http.StatusUnauthorized,
                ErrInvalidToken.Error())
            return
        }
        if !claims.HasScope(scope) {
            writeError(w, http.StatusForbidden,
                "scope "+scope+" required")
            return
        }
        next.ServeHTTP(w, r.WithContext(context.WithValue(
            r.Context(), claimsKey{}, claims)))
    })
}
```

Sumber: `transfer/internal/auth/http.go` · *tag* `lab-07`

- `POST /transfers` butuh `transfer:write`, `GET /transfers/{id}` butuh `transfer:read`
- Token tidak ada atau tidak valid: `401` dengan *header* `WWW-Authenticate`
- Token valid tanpa *scope*: `403`
- `/healthz` dan `/readyz` terbuka untuk *probe*. `/debug/circuit-breaker` terbuka hanya untuk keperluan lab dan tidak diekspos keluar di *production*

LAB 7

# *Interceptor* JWT di gRPC account

```
// GRPCScopes: GetAccount -> account:read,
// Debit dan Credit -> account:write
return func(ctx context.Context, req any,
    info *grpc.UnaryServerInfo,
    handler grpc.UnaryHandler) (any, error) {
    if public(info.FullMethod) {
        return handler(ctx, req)
    }
    scope, ok := scopes[info.FullMethod]
    if !ok {
        return nil, status.Error(codes.PermissionDenied,
            "method not allowed")
    }
    token, err := bearerFromMetadata(ctx)
    // err: Unauthenticated, missing bearer token
    claims, err := v.Verify(token)
    if err != nil {
        return nil, status.Error(codes.Unauthenticated,
            ErrInvalidToken.Error())
    }
    if !claims.HasScope(scope) {
        return nil, status.Errorf(codes.PermissionDenied,
            "scope %s required", scope)
    }
    return handler(context.WithValue(ctx,
        claimsKey{}, claims), req)
}
```

Sumber: `account/internal/auth/grpc.go` · *tag* `lab-07`

- Token dibaca dari *metadata* `authorization`
- Fungsi `public` meloloskan *method* di luar `AccountService` (*health*, *reflection*) tanpa token
- *Method* `AccountService` tanpa *scope* di *map* `scopes` ditolak (*fail closed*)
- Kode gRPC: `Unauthenticated` dan `PermissionDenied`
- *Interceptor* ini hanya menangani panggilan *unary*, sehingga RPC *streaming* dan service lain yang kelak didaftarkan tidak terlindungi

LAB 7

# Token dikirim otomatis oleh klien gRPC

```
tokenCfg := clientcredentials.Config{
    ClientID:     cfg.OAuthClientID,
    ClientSecret: cfg.OAuthClientSecret,
    TokenURL:     cfg.OAuthTokenURL,
    Scopes:       []string{"account:write"},
}
accounts, err := accountclient.New(cfg.AccountGRPCAddr,
    accountclient.ProductionSettings, tokenCfg.Token, log)

func (c *tokenCredentials) GetRequestMetadata(
    ctx context.Context, _ ...string) (map[string]string, error) {
    c.mu.Lock()
    defer c.mu.Unlock()
    if !c.tok.Valid() {
        tok, err := c.fetch(ctx)
        if err != nil {
            return nil, &tokenFetchError{err: err}
        }
        c.tok = tok
    }
    return map[string]string{"authorization":
        c.tok.Type() + " " + c.tok.AccessToken}, nil
}

// Lab tanpa TLS. Production: TLS dan return true.
func (*tokenCredentials) RequireTransportSecurity() bool { return false }
```

Sumber: `transfer/main.go`, `transfer/internal/accountclient/credentials.go` · *tag* `lab-07`

- Transfer meminta *scope* `account:write` saja, sesuai kebutuhannya (*least privilege*)
- `tokenCredentials` menyimpan token hingga 10 detik sebelum kedaluwarsa, lalu meminta token baru ke Hydra
- Satu *cache* token digunakan bersama oleh koneksi utama dan koneksi *reversal*
- Di *production*, koneksi gRPC wajib TLS dan `RequireTransportSecurity` mengembalikan `true`

LAB 7

# Token yang ditolak account diganti satu kali

```go
func (c *tokenCredentials) refreshOnUnauthenticated(
    ctx context.Context, method string, req, reply any,
    cc *grpc.ClientConn, invoker grpc.UnaryInvoker,
    opts ...grpc.CallOption) error {
    err := invoker(ctx, method, req, reply, cc, opts...)
    var fetchErr *tokenFetchError
    if status.Code(err) != codes.Unauthenticated ||
        errors.As(err, &fetchErr) {
        return err
    }
    c.mu.Lock()
    c.tok = nil
    c.mu.Unlock()
    return invoker(ctx, method, req, reply, cc, opts...)
}

// dial(): untuk koneksi utama dan koneksi reversal
grpc.WithPerRPCCredentials(tokens),
grpc.WithChainUnaryInterceptor(
    tokens.refreshOnUnauthenticated, resilience),
```

Sumber: `transfer/internal/accountclient/credentials.go`, `transfer/internal/accountclient/client.go` · *tag* `lab-07`

- Token dapat ditolak sebelum kedaluwarsa, misalnya karena Hydra mengganti *signing key*. Bila itu terjadi, token di *cache* dibuang dan panggilan diulang sekali dengan token baru
- Bila token gagal diambil (Hydra mati), klien menghasilkan `tokenFetchError` dengan kode `Unauthenticated`. Panggilan tidak sampai ke account, tidak diulang, dan tidak dihitung oleh *circuit breaker*, sehingga transfer berakhir `FAILED` dengan `502`
- *Interceptor* ini dipasang paling luar, sebelum *policy resiliency*, di kedua koneksi

ARSITEKTUR

# Posisi API *gateway*




## Tugas *gateway*

- Satu pintu masuk dari luar
- Validasi awal token, *rate limiting*, TLS
- Menyembunyikan alamat service internal

## Tetap dikerjakan service

- Validasi JWT dan *scope* di setiap service
- *Gateway* tidak menggantikan pemeriksaan di service, karena panggilan antarservice tidak melewati *gateway*

**CATATAN**

Studi kasus tidak menjalankan *gateway*. Pembahasan ini menunjukkan letaknya pada arsitektur *production*.

SECURITY

# Alasan kegagalan untuk klien dibuat generik

### Sebelum Lab 7

```
{"status": "FAILED",
 "failure_reason": "debit failed: debit
  1111111111: rpc error: code = Unavailable
  desc = dial tcp 10.0.3.7:9091: ..."}
```

Klien dapat membaca alamat internal dan detail *error* gRPC.

### Sesudah Lab 7

```
{"status": "FAILED",
 "failure_reason": "debit failed:
  account service unavailable"}
```

Detail lengkap tetap ditulis ke log service bersama `transfer_id`. Transfer yang tersimpan sebelum Lab 7 tetap memuat alasan lama.

**TIPS**

Klien cukup tahu kategori kegagalan. Penyebabnya ditelusuri dari log, dan sesi *observability* di akhir hari ini menambahkan *correlation ID* serta *trace* untuk mempercepat penelusuran tersebut.

LAB 7

# Transfer dan account diamankan dengan JWT

## Tujuan

Account hanya melayani panggilan dengan token yang valid dan *scope* yang sesuai, dan REST transfer hanya melayani aplikasi kanal yang memegang token.

## Yang dikerjakan

- Hydra dan *client* dari konfigurasi yang sudah disediakan
- *Interceptor* JWT di account
- Token *client credentials* di klien gRPC transfer
- *Middleware* JWT di REST transfer
- *Test*: token valid, kedaluwarsa, *scope* salah, *issuer* salah

- Konfigurasi wajib berupa `OAUTH_ISSUER`, `OAUTH_JWKS_URL`, `OAUTH_CLIENT_ID`, `OAUTH_CLIENT_SECRET`, dan `OAUTH_TOKEN_URL`. Service berhenti saat *startup* bila salah satunya kosong
- `make token` mencetak token `channel-app` untuk dicoba dengan curl

**TIPS**

Langkah lengkap ada di Panduan Lab, bab Lab 7. Lab ini dimulai dari tag `lab-06` dan berakhir di tag `lab-07`.

05

# *Observability* lintas service

Log, metrik, dan *trace* di kode dan di platform sebagai bekal Lab 8 di awal Hari 4

KONSEP

# Tiga pilar *observability*

| Pilar | Pertanyaan yang dijawab | Implementasi di studi kasus | Disimpan di |
|---|---|---|---|
| Log | Apa yang terjadi pada satu *request* tertentu? | `slog` format JSON, `correlation_id`, `trace_id` | Loki |
| Metrik | Seberapa sering, seberapa cepat, berapa yang gagal? | `prometheus/client_golang`, *endpoint* `/metrics` | Prometheus |
| *Trace* | Di service mana sebagian besar waktu satu transaksi terpakai? | OpenTelemetry, `otelhttp`, `otelgrpc`, *header* Kafka | Tempo |

- Pada monolith, satu *stack trace* cukup untuk menjelaskan kegagalan. Pada microservices, satu transfer melewati tiga service dan sebuah *broker*. Ketiga pilar ditampilkan di Grafana.
- Metrik mendeteksi adanya masalah. *Trace* menunjukkan service yang paling banyak menyita waktu, dan log berisi penyebabnya.
- *Trace* dan log dihubungkan oleh `trace_id`. Dari lonjakan di grafik metrik, *engineer* mempersempit rentang waktu lalu membuka *trace* pada rentang tersebut. Tautan langsung dari metrik ke *trace* membutuhkan *exemplar* Prometheus, dan fitur tersebut tidak dipasang di lab.

PETA CONCERN MICROSERVICES

# *Observability* di kode dan di platform

## Di kode setiap service (Lab 8)

- **Log.** slog dengan *handler* JSON ke stdout
- **Metrik.** Prometheus client (`client_golang`), dibaca dari `GET /metrics`
- **Trace.** OpenTelemetry SDK: otelhttp dan otelgrpc, *trace context* di *header* Kafka ditulis sendiri, dikirim melalui OTLP
- Setiap service punya salinan paket `telemetry` sendiri. Yang dibagi hanya konvensi, misalnya nama *header* dan nama metrik

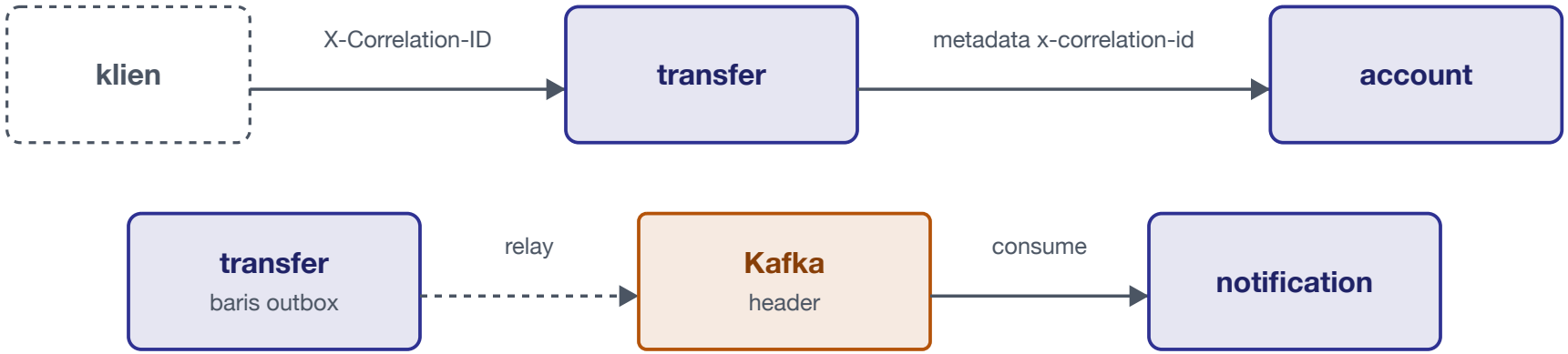
## Di platform

- **Prometheus** mengambil (*scrape*) `/metrics` secara berkala
- **Loki** menyimpan log, **Tempo** menyimpan *trace*
- **Grafana** menampilkan ketiganya dan menghubungkan log dengan *trace*
- Lab menggunakan `otel-lgtm`, yaitu semua komponen di atas dalam satu *container*. *Production* menjalankannya terpisah dengan *storage* (Hari 4)

CATATAN

**12-factor, faktor XI (*logs*).** Service menulis log sebagai aliran event ke stdout, tanpa mengatur *file* atau rotasi. Pengumpulan dan penyimpanan log menjadi tugas platform, misalnya `kubectl logs` dan *log collector* di Kubernetes. Daftar 12-factor diperkenalkan di Hari 1, sesi Anatomi Service Go. Di Lab 8, catatan log yang sama juga dikirim melalui OTLP agar langsung tampil di Loki.

# *Correlation ID* lintas tiga service




- Transfer membaca *header* `X-Correlation-ID`, atau membuat ID baru bila *header* tersebut kosong, lalu mengembalikannya di *response*.
- ID disimpan di `context.Context`, ikut ke gRPC *metadata*, ke baris *outbox*, lalu ke *header* Kafka.

# *Handler* `slog` yang membaca `context`

```go
type ctxHandler struct{ slog.Handler }

func (h ctxHandler) Handle(ctx context.Context,
    r slog.Record) error {
    if id, ok := correlation.FromContext(ctx); ok {
        r.AddAttrs(slog.String("correlation_id", id))
    }
    sc := trace.SpanContextFromContext(ctx)
    if sc.IsValid() {
        r.AddAttrs(
            slog.String("trace_id", sc.TraceID().String()),
            slog.String("span_id", sc.SpanID().String()))
    }
    return h.Handler.Handle(ctx, r)
}

func (h ctxHandler) WithAttrs(a []slog.Attr) slog.Handler {
    return ctxHandler{h.Handler.WithAttrs(a)}
}

func (h ctxHandler) WithGroup(n string) slog.Handler {
    return ctxHandler{h.Handler.WithGroup(n)}
}
```

Sumber: `transfer/internal/telemetry/log.go` · *tag* `lab-08`

- Setiap baris *log* otomatis membawa `correlation_id`, `trace_id`, `span_id`
- Kode *handler* HTTP cukup memanggil `slog.InfoContext(ctx, ...)`
- *Output* JSON ke *stdout* sekaligus diekspor sebagai *log* OTLP (*bridge* `otelslog`)
- `WithAttrs` dan `WithGroup` membungkus ulang *handler* hasilnya, sehingga *logger* dari `logger.With(...)` tetap membawa ketiga atribut tersebut

METRIK

# Metrik yang dipasang di setiap service

| Metrik | Service | Kegunaan |
|---|---|---|
| Histogram durasi *request* (`method`, `route`, `status`) | Semua | *Latency* p95 per *endpoint* |
| `transfers_total{status}` | transfer | Laju transfer per status: `COMPLETED`, `FAILED`, `COMPENSATED` |
| `circuit_breaker_state{target}` | transfer | 0 = *closed*, 1 = *half-open*, 2 = *open* |
| *Counter* transisi *breaker* | transfer | Berapa kali *breaker* berganti status |
| `outbox_pending` | transfer | *Event* yang belum terkirim ke Kafka |
| `notifications_processed_total{result}` | notification | `sent`, `duplicate`, dan `skipped`. `skipped` menghitung *event* rusak atau yang ditolak database, dan nilainya seharusnya tetap 0 |

**PERHATIAN**

Label `route` diisi pola ServeMux (`GET /transfers/{id}`). Bila diisi *path* asli yang mengandung ID, jumlah *time series* akan terus bertambah.

# Meneruskan *trace context* lewat Kafka

```go
// telemetry.Setup: hanya W3C Trace Context
otel.SetTextMapPropagator(propagation.TraceContext{})

// transfer, outbox.Insert: dalam transaksi DB yang
// sama dengan perubahan status transfer
carrier := propagation.MapCarrier{}
otel.GetTextMapPropagator().Inject(ctx, carrier)
// carrier masuk kolom trace_context,
// relay menyalinnya ke header Kafka

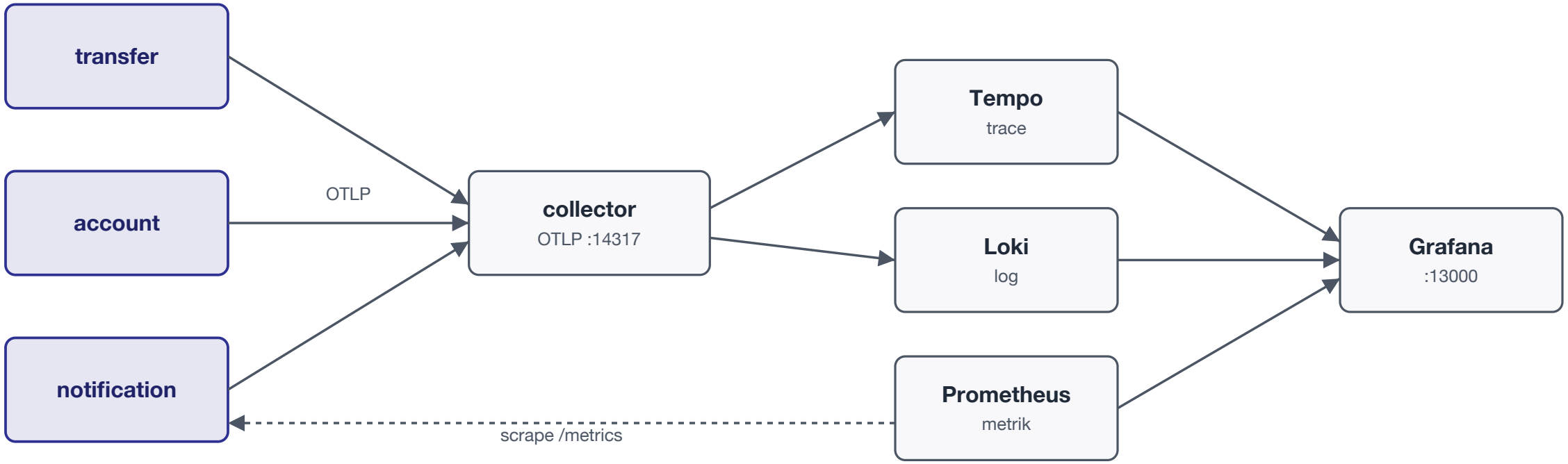
// notification: saat membaca message
headers := m.Headers
ctx = otel.GetTextMapPropagator().Extract(ctx,
    telemetry.KafkaHeaders{Headers: &headers})
ctx, span := c.tracer.Start(ctx, "process "+m.Topic,
    trace.WithSpanKind(trace.SpanKindConsumer))
defer span.End()
```

Sumber: `transfer/internal/telemetry/telemetry.go`, `transfer/internal/outbox/outbox.go`, `notification/internal/notification/consumer.go` · *tag* `lab-08`

- HTTP dan gRPC: `otelhttp` dan *stats handler* `otelgrpc` meneruskan *header* `traceparent` secara otomatis
- Kafka: *relay* berjalan di *goroutine* terpisah tanpa *context* dari *request* asal, dan baru mengirim *event* hingga ±500 ms kemudian. Karena itu *trace context* disimpan ke baris *outbox* saat baris tersebut ditulis
- Hanya `traceparent` yang diteruskan. *Baggage* tidak digunakan karena REST API diakses langsung oleh aplikasi *channel*, dan *baggage* dari pemanggil luar itu akan ikut terkirim ke semua service yang dipanggil sesudahnya
- Hasilnya, satu transfer tampil sebagai satu *trace* yang mencakup tiga service

STACK

# *Observability stack* lab dalam satu *container* `otel-lgtm`




- Satu *container* `grafana/otel-lgtm` berisi *collector*, Tempo, Loki, Prometheus, dan Grafana dengan *dashboard* bawaan.
- *Trace* dan *log* dikirim (*push*) lewat OTLP, sedangkan metrik diambil (*pull*) oleh Prometheus dari `/metrics` di setiap service.

KEPUTUSAN DESAIN

# `otel-lgtm` untuk *observability* di lab

## Kebutuhan

- Lab 8 membutuhkan *collector*, penyimpanan *trace*, *log*, metrik, dan Grafana di setiap laptop
- RAM laptop terbatas, dan waktu lab tidak cukup untuk mengonfigurasi lima komponen

## Opsi yang dipertimbangkan

- Komponen terpisah (OTel Collector, Tempo, Loki, Prometheus, Grafana) di `compose.yaml`
- Jaeger, khusus untuk *trace*
- Platform *observability* SaaS
- `grafana/otel-lgtm`: kelima komponen tersebut dalam satu *container*

## Pilihan di studi kasus

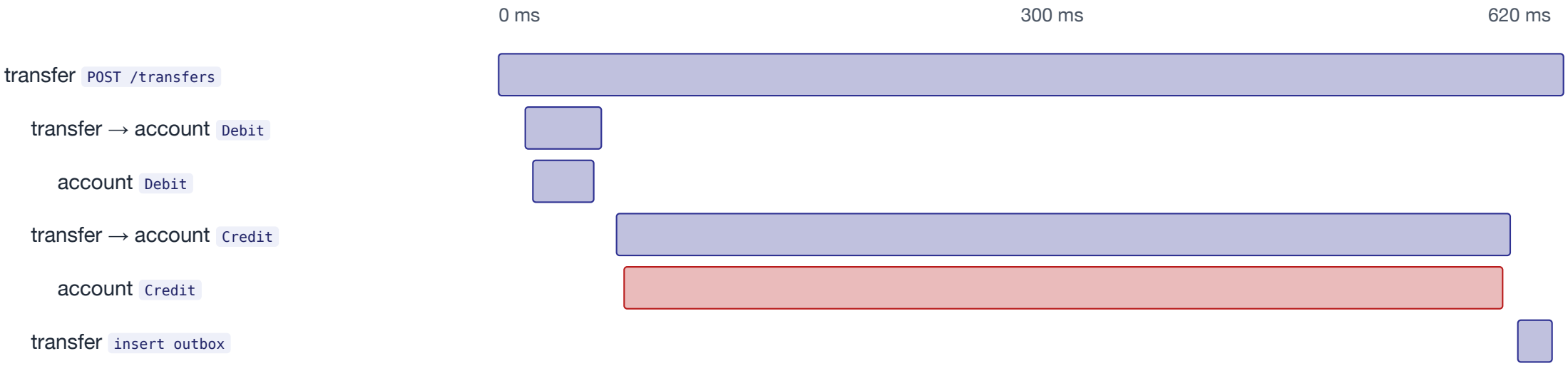
- `grafana/otel-lgtm:0.37.0` di `compose.yaml`, *port* 13000 (Grafana) dan 14317/14318 (OTLP) di laptop
- Service hanya mengenal `OTEL_EXPORTER_OTLP_ENDPOINT`

## Konsekuensi dan *trade-off*

- Ditujukan untuk pengembangan, demo, dan *testing*. Tidak ada *high availability* maupun kontrol akses per tim
- Di *production*, setiap komponen dijalankan dan diskalakan sendiri (Hari 4, bagian Operasional)
- Kode service tidak berubah saat pindah ke *stack* terpisah, cukup alamat OTLP yang diganti

DEMO GRAFANA

# Mencari service yang lambat dari *waterfall*




- Setiap baris menggambarkan satu *span*, dan baris yang menjorok adalah *child span* dari *parent span* di atasnya
- Panjang *bar* menunjukkan durasi. `Credit` di account menghabiskan hampir seluruh waktu transfer, sehingga account yang diperiksa lebih dahulu
- Penyebabnya terbaca dari atribut *span* dan dari log dengan `trace_id` yang sama, misalnya *fault delay* yang dipasang dengan `make fault-slow` di Lab 8

**TIPS**

Diagram di atas hanya ilustrasi. Demo sesi ini dijalankan instruktur dari tag `lab-08`, karena `otel-lgtm` baru ditambahkan ke `compose.yaml` di Lab 8.

DEMO GRAFANA

# Dari baris log ke *trace*

```
{"time":"2026-10-28T10:15:00.412Z","level":"WARN",
 "msg":"credit failed after debit, reversing the debit","transfer_id":"3b2a...9d",
 "correlation_id":"c7f3e1...","trace_id":"4bf92f3577b34da6a3ce929d0e0e4736",
 "span_id":"00f067aa0ba902b7","error":"rpc error: code = DeadlineExceeded"}
```

1. Di Grafana *Explore*, pilih Loki lalu *filter* log berdasarkan `correlation_id` yang dikembalikan transfer di *header* `X-Correlation-ID`
2. Semua baris log dari transfer, account, dan notification untuk *request* tersebut tampil berurutan menurut waktu
3. Nilai `trace_id` pada baris log membuka *trace* yang sama di Tempo

**CATATAN**

*Correlation ID* digunakan untuk mencari log satu *request*, sedangkan `trace_id` menghubungkan log tersebut ke *span* yang menunjukkan durasi setiap langkah.

DEMO GRAFANA

# Membaca metrik dengan PromQL

| Panel | PromQL |
|---|---|
| Laju transfer per status | `sum by (status) (rate(transfers_total[5m]))` |
| *Latency* p95 per *route* | `histogram_quantile(0.95, sum by (le, route) (rate(http_server_request_duration_seconds_bucket[5m])))` |

- `rate(...[5m])` mengubah *counter* yang terus naik menjadi laju per detik dalam rentang 5 menit
- p95 dihitung dari *bucket* histogram dan berarti 95% *request* selesai di bawah angka tersebut
- Bila `FAILED` atau `COMPENSATED` naik bersamaan dengan p95, periksa *trace* pada rentang waktu yang sama

HARI 4

# Lanjutan di Lab 8

## Yang ditambahkan ke ketiga service

- slog JSON dengan `correlation_id`, `trace_id`, `span_id`
- *Endpoint* `/metrics` untuk Prometheus
- *Trace* OpenTelemetry melalui OTLP
- *Dashboard* Grafana yang sudah disediakan

## Latihan

- Beri *delay* pada account dengan `make fault-slow`
- Temukan *span* yang lambat di Tempo
- Baca status *circuit breaker* dari metrik

**TIPS**

Event dan outbox yang dibuat hari ini menjadi bagian *trace* paling menarik di Lab 8, karena *trace* tetap tersambung walaupun melewati Kafka.

RINGKASAN

# Pola hari ini dan syarat penggunaannya

| Pola | Masalah yang diselesaikan | Layak digunakan ketika |
|---|---|---|
| *Transactional outbox* | Penulisan ke DB dan *publish* event tidak atomik | Event wajib terkirim jika dan hanya jika data tersimpan |
| *Idempotent consumer* | Duplikat dari *at-least-once delivery* | *Consumer* mengubah data atau memicu efek ke luar |
| *Idempotency-Key* | Klien mengirim ulang *request* | *Endpoint* membuat transaksi finansial |
| OAuth2 *client credentials* + JWT | Panggilan antarservice tanpa identitas | Setiap service internal yang mengubah data |
| *Correlation ID + tracing* | Satu transaksi tersebar di banyak log | Satu *request* melewati lebih dari satu service |

PETA CONCERN MICROSERVICES

# Posisi Hari 3 pada peta *concern*

| # | ***Concern*** | **Kondisi studi kasus setelah Lab 7** | **Berikutnya** |
|---|---|---|---|
| 1 | *Decomposition* dan kepemilikan data | Tiga service, masing-masing dengan database sendiri | — |
| **2** | **Komunikasi sinkron dan asinkron** | REST dari kanal, gRPC transfer ke account, event Kafka ke notification | — |
| 3 | *Resiliency* | failsafe-go di klien gRPC, *reversal* dengan *policy* sendiri | *Probe* dan *rolling update* (Lab 9) |
| **4** | **Konsistensi data** | Saga dengan kompensasi, outbox, *idempotent consumer*, *Idempotency-Key* | *Recovery job* di *production* |
| **5** | ***Security*** | JWT di REST transfer dan gRPC account, token dari Hydra | TLS dan API *gateway* di *production* |
| **6** | ***Observability*** | Konsep dan rancangan instrumentasi sudah dibahas, kodenya dipasang di Lab 8 | Lab 8 di awal Hari 4 |
| 7 | *Configuration* (termasuk *centralized config* dan *secret*) | *Environment variable*, service berhenti bila nilai wajib kosong | ConfigMap dan Secret (Lab 9) |
| 8 | *Service discovery* dan *load balancing* | Alamat `localhost` dari konfigurasi | Kubernetes Service (Lab 9) |
| 9 | *Packaging* dan *deployment* | Proses Go di laptop | *Container* dan k3d (Lab 9) |
| 10 | *Automation* (CI/CD) | — | *Pipeline* (Lab 10) |

PENUTUP

# Pertanyaan untuk direnungkan

- Pada sistem yang sedang dikerjakan, adakah proses yang menulis ke database lalu mengirim event atau memanggil sistem lain tanpa outbox?
- Saat *timeout* terjadi, apakah pemanggil tahu permintaannya sudah diproses atau belum? Apa penanda keunikannya?
- Siapa yang menangani transfer berstatus `COMPENSATION_FAILED`, dan dari mana petugas tersebut tahu ada kasus baru?
- Service internal mana yang saat ini bisa dipanggil tanpa token?

> **CATATAN**
>
> *Checkpoint* hari ini: tag `lab-05`, `lab-06`, dan `lab-07` di repositori studi kasus. Peserta yang tertinggal dapat melanjutkan dari tag lab sebelumnya.